

Labo 05 — Questions

Question 1 [Analyse]

Une équipe livre son API Spring Boot dans une image de 950 Mo construite à partir de `maven:3.9-eclipse-temurin-21`. Le responsable sécurité refuse la mise en production. Donnez **trois** arguments qui n'ont rien à voir avec l'espace disque, puis dites lequel est le plus difficile à corriger autrement que par un multi-stage.

Question 2 [Compréhension]

Dans un Dockerfile multi-stage à trois `FROM`, lequel produit l'image finale ? Que deviennent les autres ? Et si aucun `COPY --from` ne référence le deuxième stage, que fait BuildKit ?

Question 3 [Diagnostic]

Un développeur écrit :

```
FROM maven:3.9-eclipse-temurin-21 AS build
COPY . /app
WORKDIR /app
RUN mvn package -DskipTests

FROM eclipse-temurin:21-jre-alpine
COPY --from=build /app /app
ENTRYPOINT ["java", "-jar", "/app/target/api.jar"]
```

L'image finale fait 420 Mo au lieu des 250 Mo attendus, et le code source s'y trouve toujours. Expliquez l'erreur et corrigez-la en une ligne.

Question 4 [Analyse]

Votre collègue veut « gagner du temps » en conteneurisant `ng serve` : l'image contient Node, le projet source, et démarre le serveur de développement Angular sur le port 4200. Ça fonctionne en recette. Donnez quatre raisons de refuser cette image en production, puis décrivez en deux phrases ce qu'il faut faire à la place.

Question 5 [Analyse]

Une équipe migre son image de `eclipse-temurin:21-jre-jammy` (Debian) vers `eclipse-temurin:21-jre-alpine` et gagne 130 Mo. Deux semaines plus tard, un traitement de génération de PDF échoue en production avec une `UnsatisfiedLinkError`. Expliquez le lien probable, et dites comment cette migration aurait dû être menée.

Question 6 [Compréhension]

Pourquoi dit-on que le multi-stage est la seule protection **fiable** contre les secrets de build, alors qu'on peut aussi supprimer le fichier avec un `rm` ? Dans quel cas le multi-stage ne protège-t-il pas ?

Question 7 [Analyse]

Comparez ces deux stratégies pour un JAR Spring Boot de 50 Mo, du point de vue du **temps de déploiement** d'un correctif d'une ligne : (a) `COPY target/api.jar app.jar`, (b) l'extraction en

couches (`layertools`). Chiffrez approximativement ce qui est transféré dans chaque cas, et dites pourquoi (a) reste acceptable dans beaucoup d'entreprises.

Question 8 [Diagnostic]

Un build de CI passe de 90 secondes à 7 minutes après le passage à un nouvel agent, sans qu'aucun fichier n'ait changé. Le Dockerfile est correctement ordonné (dépendances avant code). Expliquez, et donnez deux mécanismes qui permettent de retrouver les 90 secondes.

Question 9 [Analyse]

Une image *distroless* réduit fortement la surface d'attaque. Citez deux capacités d'exploitation que vous perdez concrètement, et dites comment une équipe compense habituellement chacune.

Question 10 [Compréhension]

`RUN --mount=type=cache,target=/root/.m2 mvn package` : où sont stockées ces données, et pourquoi n'apparaissent-elles pas dans l'image finale ? En quoi est-ce différent d'un `VOLUME` ?

Question 11 [Analyse]

Un développeur affirme : « Le multi-stage ne sert à rien pour Angular, puisque de toute façon le résultat n'est que des fichiers statiques. » Répondez-lui en décrivant ce que contiendrait l'image sans multi-stage, et l'écart de taille en jeu.

Question 12 [Diagnostic]

Le Dockerfile suivant échoue avec `COPY failed: stat /app/dist: no such file or directory` :

```
FROM node:22-alpine AS build
WORKDIR /src
COPY . .
RUN npm ci && npm run build

FROM nginx:alpine
COPY --from=build /app/dist /usr/share/nginx/html
```

Trouvez l'erreur, puis dites quelle commande vous permettrait d'inspecter le contenu réel du stage `build` pour la diagnostiquer sans deviner.

Question 13 [Analyse]

Votre entreprise impose que les tests unitaires fassent échouer la construction de l'image. Où placez-vous `RUN mvn test` dans un Dockerfile multi-stage, et quel est l'inconvénient de cette approche par rapport à des tests exécutés en amont par la CI ?

Question 14 [Analyse]

Deux images de la même application : l'une de 250 Mo en une seule couche, l'autre de 280 Mo en cinq couches dont quatre stables. Laquelle se déploie le plus vite lors d'une mise à jour du code applicatif ? Justifiez, et dites dans quelle situation la réponse s'inverse.